a. Show how to convert an equality constraint into an equivalent set of inequalities. That is, given a constraint $\sum_{j=1}^{n} a_{ij}x_j = b_i$, give a set of inequalities that will be satisfied if and only if $\sum_{j=1}^{n} a_{ij}x_j = b_i$,

b. Show how to convert an inequality constraint $\sum_{j=1}^{n} a_{ij}x_j \le b_i$ into an equality constraint and a nonnegativity constraint. You will need to introduce an additional variable s, and use the constraint that $s \ge 0$.

29.1-7

Explain how to convert a minimization linear program to an equivalent maximization linear program, and argue that your new linear program is equivalent to the original one.

29.1-8

In the political problem at the beginning of this chapter, there are feasible solutions that correspond to winning more voters than there actually are in the district. For example, you can set x_2 to 200, x_3 to 200, and $x_1 = x_4 = 0$. That solution is feasible, yet it seems to say that you will win 400,000 suburban voters, even though there are only 200,000 actual suburban voters. What constraints can you add to the linear program to ensure that you never seem to win more voters than there actually are? Even if you don't add these constraints, argue that the optimal solution to this linear program can never win more voters than there actually are in the district.

29.2 Formulating problems as linear programs

Linear programming has many applications. Any textbook on operations research is filled with examples of linear programming, and linear programming has become a standard tool taught to students in most business schools. The election scenario is one typical example. Here are two more examples:

- An airline wishes to schedule its flight crews. The Federal Aviation Administration imposes several constraints, such as limiting the number of consecutive hours that each crew member can work

and insisting that a particular crew work only on one model of aircraft during each month. The airline wants to schedule crews on all of its flights using as few crew members as possible.

- An oil company wants to decide where to drill for oil. Siting a drill at a particular location has an associated cost and, based on geological surveys, an expected payoff of some number of barrels of oil. The company has a limited budget for locating new drills and wants to maximize the amount of oil it expects to find, given this budget.

Linear programs also model and solve graph and combinatorial problems, such as those appearing in this book. We have already seen a special case of linear programming used to solve systems of difference constraints in Section 22.4. In this section, we'll study how to formulate several graph and network-flow problems as linear programs. Section 35.4 uses linear programming as a tool to find an approximate solution to another graph problem.

Perhaps the most important aspect of linear programming is to be able to recognize when you can formulate a problem as a linear program. Once you cast a problem as a polynomial-sized linear program, you can solve it in polynomial time by the ellipsoid algorithm or interior-point methods. Several linear-programming software packages can solve problems efficiently, so that once the problem is in the form of a linear program, such a package can solve it.

We'll look at several concrete examples of linear-programming problems. We start with two problems that we have already studied: the single-source shortest-paths problem from Chapter 22 and the maximum-flow problem from Chapter 24. We then describe the minimum-cost-flow problem. (Although the minimum-cost-flow problem has a polynomial-time algorithm that is not based on linear programming, we won't describe the algorithm.) Finally, we describe the multicommodity-flow problem, for which the only known polynomial-time algorithm is based on linear programming.

When we solved graph problems in Part VI, we used attribute notation, such as $v.d$ and $(u, v).f$. Linear programs typically use subscripted variables rather than objects with attached attributes,

however. Therefore, when we express variables in linear programs, we indicate vertices and edges through subscripts. For example, we denote the shortest-path weight for vertex v not by $v.d$ but by d_v, and we denote the flow from vertex u to vertex v not by $(u, v).f$ but by f_{uv}. For quantities that are given as inputs to problems, such as edge weights or capacities, we continue to use notations such as $w(u, v)$ and $c(u, v)$.

Shortest paths

We can formulate the single-source shortest-paths problem as a linear program. We'll focus on how to formulate the single-pair shortest-path problem, leaving the extension to the more general single-source shortest-paths problem as Exercise 29.2-2.

In the single-pair shortest-path problem, the input is a weighted, directed graph $G = (V, E)$, with weight function $w : E \to \mathbb{R}$ mapping edges to real-valued weights, a source vertex s, and destination vertex t. The goal is to compute the value d_t, which is the weight of a shortest path from s to t. To express this problem as a linear program, you need to determine a set of variables and constraints that define when you have a shortest path from s to t. The triangle inequality (Lemma 22.10 on page 633) gives $d_v \le d_u + w(u, v)$ for each edge $(u, v) \in E$. The source vertex initially receives a value $d_s = 0$, which never changes. Thus the following linear program expresses the shortest-path weight from s to t:

$$\text{maximize} \quad d_t \tag{29.22}$$

subject to

$$d_v \le d_u + w(u, v) \quad \text{for each edge } (u, v) \in E \tag{29.23}$$

$$d_s = 0 \ . \tag{29.24}$$

You might be surprised that this linear program maximizes an objective function when it is supposed to compute shortest paths. Minimizing the objective function would be a mistake, because when all the edge weights are nonnegative, setting $\overline{d}_v = 0$ for all $v \in V$ (recall that a bar over a variable name denotes a specific setting of the variable's value) would yield an optimal solution to the linear program without solving

the shortest-paths problem. Maximizing is the right thing to do because an optimal solution to the shortest-paths problem sets each $\overline{d}_v$ to min $\{\overline{d}_u + w(u, v) : u \in V \text{ and } (u, v) \in E\}$, so that $\overline{d}_v$ is the largest value that is less than or equal to all of the values in the set $\{\overline{d}_u + w(u, v)$. Therefore, it makes sense to maximize d_v for all vertices v on a shortest path from s to t subject to these constraints, and maximizing d_t achieves this goal.

This linear program has $|V|$ variables d_v, one for each vertex $v \in V$. It also has $|E| + 1$ constraints: one for each edge, plus the additional constraint that the source vertex's shortest-path weight always has the value 0.

Maximum flow

Next, let's express the maximum-flow problem as a linear program. Recall that the input is a directed graph $G = (V, E)$ in which each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$, and two distinguished vertices: a source s and a sink t. As defined in Section 24.1, a flow is a nonnegative real-valued function $f : V \times V \to \mathbb{R}$ that satisfies the capacity constraint and flow conservation. A maximum flow is a flow that satisfies these constraints and maximizes the flow value, which is the total flow coming out of the source minus the total flow into the source. A flow, therefore, satisfies linear constraints, and the value of a flow is a linear function. Recalling also that we assume that $c(u, v) = 0$ if $(u, v) \notin E$ and that there are no antiparallel edges, the maximum-flow problem can be expressed as a linear program:

$$\text{maximize} \quad \sum_{v \in V} f_{sv} - \sum_{v \in V} f_{vs} \tag{29.25}$$

subject to

$$f_{uv} \leq c(u, v) \quad \text{for each } u, v \in V \tag{29.26}$$

$$\sum_{v \in V} f_{vu} = \sum_{v \in V} f_{uv} \quad \text{for each } u \in V - \{s, t\} \tag{29.27}$$

$$f_{uv} \geq 0 \quad \text{for each } u, v \in V \ . \tag{29.28}$$

This linear program has $|V|^2$ variables, corresponding to the flow between each pair of vertices, and it has $2|V|^2 + |V| - 2$ constraints.

It is usually more efficient to solve a smaller-sized linear program. The linear program in (29.25)–(29.28) has, for ease of notation, a flow and capacity of 0 for each pair of vertices u, v with $(u, v) \notin E$. It is more efficient to rewrite the linear program so that it has $O(V + E)$ constraints. Exercise 29.2-4 asks you to do so.

Minimum-cost flow

In this section, we have used linear programming to solve problems for which we already knew efficient algorithms. In fact, an efficient algorithm designed specifically for a problem, such as Dijkstra's algorithm for the single-source shortest-paths problem, will often be more efficient than linear programming, both in theory and in practice.

The real power of linear programming comes from the ability to solve new problems. Recall the problem faced by the politician in the beginning of this chapter. The problem of obtaining a sufficient number of votes, while not spending too much money, is not solved by any of the algorithms that we have studied in this book, yet it can be solved by linear programming. Books abound with such real-world problems that linear programming can solve. Linear programming is also particularly useful for solving variants of problems for which we may not already know of an efficient algorithm.

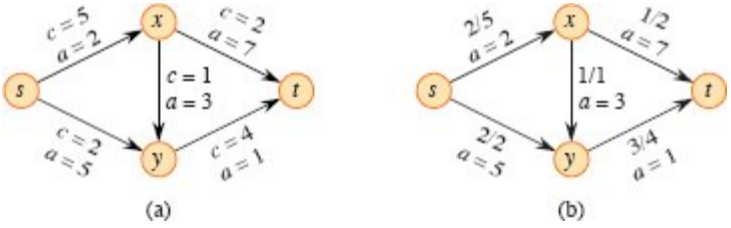

Figure 29.3 (a) An example of a minimum-cost-flow problem. Capacities are denoted by c and costs by a. Vertex s is the source, and vertex t is the sink. The goal is to send 4 units of flow from s to t. **(b)** A solution to the minimum-cost flow problem in which 4 units of flow are sent from s to t. For each edge, the flow and capacity are written as flow/capacity.

Consider, for example, the following generalization of the maximum-flow problem. Suppose that, in addition to a capacity $c(u, v)$ for each edge (u, v), you are given a real-valued cost $a(u, v)$. As in the maximum-flow problem, assume that $c(u, v) = 0$ if $(u, v) \notin E$ and that there are no

antiparallel edges. If you send f_{uv} units of flow over edge (u, v), you incur a cost of $a(u, v) \cdot f_{uv}$. You are also given a flow demand d. You wish to send d units of flow from s to t while minimizing the total cost $\sum_{(u,v)\in E} a(u, v) \cdot f_{uv}$ incurred by the flow. This problem is known as the ***minimum-cost-flow problem***.

Figure 29.3(a) shows an example of the minimum-cost-flow problem, with a goal of sending 4 units of flow from s to t while incurring the minimum total cost. Any particular legal flow, that is, a function f satisfying constraints (29.26)–(29.28), incurs a total cost of $\sum_{(u,v)\in E} a(u, v) \cdot f_{uv}$. What is the particular 4-unit flow that minimizes this cost? Figure 29.3(b) shows an optimal solution, with total cost $\sum_{(u,v)\in E} a(u, v) \cdot f_{uv} = (2 \cdot 2) + (5 \cdot 2) + (3 \cdot 1) + (7 \cdot 1) + (1 \cdot 3) = 27$.

There are polynomial-time algorithms specifically designed for the minimum-cost-flow problem, but they are beyond the scope of this book. The minimum-cost-flow problem can be expressed as a linear program, however. The linear program looks similar to the one for the maximum-flow problem with the additional constraint that the value of the flow must be exactly d units, and with the new objective function of minimizing the cost:

$$\text{minimize} \quad \sum_{(u,v)\in E} a(u, v) \cdot f_{uv} \tag{29.29}$$

subject to

$$\begin{aligned}
f_{uv} &\le c(u, v) && \text{for each } u, v \in V \\
\sum_{v\in V} f_{vu} - \sum_{v\in V} f_{uv} &= 0 && \text{for each } u \in V - \{s, t\} \\
\sum_{v\in V} f_{sv} - \sum_{v\in V} f_{vs} &= d \\
f_{uv} &\ge 0 && \text{for each } u, v \in V \ .
\end{aligned} \tag{29.30}$$

Multicommodity flow

As a final example, let's consider another flow problem. Suppose that the Lucky Puck company from Section 24.1 decides to diversify its product line and ship not only hockey pucks, but also hockey sticks and hockey helmets. Each piece of equipment is manufactured in its own

factory, has its own warehouse, and must be shipped, each day, from factory to warehouse. The sticks are manufactured in Vancouver and are needed in Saskatoon, and the helmets are manufactured in Edmonton and must be shipped to Regina. The capacity of the shipping network does not change, however, and the different items, or ***commodities***, must share the same network.

This example is an instance of a ***multicommodity-flow problem***. The input to this problem is once again a directed graph $G = (V, E)$ in which each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$. As in the maximum-flow problem, implicitly assume that $c(u, v) = 0$ for $(u, v) \notin E$ and that the graph has no antiparallel edges. In addition, there are k different commodities, $K_1, K_2, \ldots, K_k$, with commodity i specified by the triple $K_i = (s_i, t_i, d_i)$. Here, vertex s_i is the source of commodity i, vertex t_i is the sink of commodity i, and d_i is the demand for commodity i, which is the desired flow value for the commodity from s_i to t_i. We define a flow for commodity i, denoted by f_i, (so that f_{iuv} is the flow of commodity i from vertex u to vertex v) to be a real-valued function that satisfies the flow-conservation and capacity constraints. We define f_{uv}, the ***aggregate flow***, to be the sum of the various commodity flows, so that $f_{uv} = \sum_{i=1}^{k} f_{iuv}$. The aggregate flow on edge (u, v) must be no more than the capacity of edge (u, v). This problem has no objective function: the question is to determine whether such a flow exists. Thus the linear program for this problem has a "null" objective function:

$$
\begin{aligned}
&\text{minimize} && 0 && \\
&\text{subject to} && && \\
& && \sum_{i=1}^{k} f_{iuv} \leq c(u,v) && \text{for each } u, v \in V \\
& \sum_{v \in V} f_{iuv} - \sum_{v \in V} f_{ivu} = 0 && && \text{for each } i = 1, 2, \ldots, k \text{ and} \\
& && && \text{for each } u \in V - \{s_i, t_i\} \\
& \sum_{v \in V} f_{i,s_i,v} - \sum_{v \in V} f_{i,v,s_i} = d_i && && \text{for each } i = 1, 2, \ldots, k \\
& && f_{iuv} \geq 0 && \text{for each } u, v \in V \text{ and} \\
& && && \text{for each } i = 1, 2, \ldots, k \ .
\end{aligned}
$$

The only known polynomial-time algorithm for this problem expresses it as a linear program and then solves it with a polynomial-time linear-

programming algorithm.

Exercises

29.2-1
Write out explicitly the linear program corresponding to finding the shortest path from vertex s to vertex x in Figure 22.2(a) on page 609.

29.2-2
Given a graph G, write a linear program for the single-source shortest-paths problem. The solution should have the property that d_v is the shortest-path weight from the source vertex s to v for each vertex $v \in V$.

29.2-3
Write out explicitly the linear program corresponding to finding the maximum flow in Figure 24.1(a).

29.2-4
Rewrite the linear program for maximum flow (29.25)–(29.28) so that it uses only $O(V + E)$ constraints.

29.2-5
Write a linear program that, given a bipartite graph $G = (V, E)$, solves the maximum-bipartite-matching problem.

29.2-6
There can be more than one way to model a particular problem as a linear program. This exercise gives an alternative formulation for the maximum-flow problem. Let $P = \{P_1, P_2, \ldots, P_p\}$ be the set of *all* possible directed simple paths from source s to sink t. Using decision variables $x_1, \ldots, x_p$, where x_i is the amount of flow on path i, formulate a linear program for the maximum-flow problem. What is an upper bound on p, the number of directed simple paths from s to t?

29.2-7
In the ***minimum-cost multicommodity-flow problem***, the input is a directed graph $G = (V, E)$ in which each edge $(u, v) \in E$ has a

nonnegative capacity $c(u, v) \geq 0$ and a cost $a(u, v)$. As in the multicommodity-flow problem, there are k different commodities, K_1, K_2, … , K_k, with commodity i specified by the triple $K_i = (s_i, t_i, d_i)$. We define the flow f_i for commodity i and the aggregate flow f_{uv} on edge (u, v) as in the multicommodity-flow problem. A feasible flow is one in which the aggregate flow on each edge (u, v) is no more than the capacity of edge (u, v). The cost of a flow is $\sum_{u,v \in E} a(u, v) \cdot f_{uv}$, and the goal is to find the feasible flow of minimum cost. Express this problem as a linear program.

29.3 Duality

We will now introduce a powerful concept called ***linear-programming duality***. In general, given a maximization problem, duality allows you to formulate a related minimization problem that has the same objective value. The idea of duality is actually more general than linear programming, but we restrict our attention to linear programming in this section.

Duality enables us to prove that a solution is indeed optimal. We saw an example of duality in Chapter 24 with Theorem 24.6, the max-flow min-cut theorem. Suppose that, given an instance of a maximum-flow problem, you find a flow f with value $|f|$. How do you know whether f is a maximum flow? By the max-flow min-cut theorem, if you can find a cut whose value is also $|f|$, then you have verified that f is indeed a maximum flow. This relationship provides an example of duality: given a maximization problem, define a related minimization problem such that the two problems have the same optimal objective values.

Given a linear program in standard form in which the objective is to maximize, let's see how to formulate a ***dual*** linear program in which the objective is to minimize and whose optimal value is identical to that of the original linear program. When referring to dual linear programs, we call the original linear program the ***primal***.

Given the primal linear program